D01 · 기획서

Convoy

superpowers 계획서의 독립 Task를 여러 작업자에게 동시에 맡기고, 조율은 코드 하나가 맡는 실행 엔진

문서 버전	1.0
작성일	2026-10-20
작성자	김태민
문서 상태	기획 검토본

변경 이력

버전	일자	내용	작성자
1.0	2026-10-20	최초 작성	김태민

목차

1. 배경과 문제 정의
2. 현재 대안과 한계
3. 대상 사용자 (페르소나)
4. 핵심 가치
5. 사용 흐름
6. MVP 범위와 제외
7. 성공 지표
8. 리스크와 대응
9. 로드맵

부록 A. 사용자 인터뷰 가이드 · 부록 B. 인터뷰 결과 기록 양식

표기 원칙 — 가설 검증 전 추정 · 목표치 측정 전 목표 · 구현 1단계 확인 필요 `claude -p` 동작 중 아직 실측하지 않은 것. 이 표시가 없는 사실 진술은 출처가 있거나 직접 확인한 것이다. 문서 간 참조는 공통 ID(`CMD-nn` 명령 · `SN-nn` 장면 · `POL-nn` 정책 · `ADR-nn` 결정 · `ST-T-nn` / `ST-C-nn` 상태)로 하며, 끊긴 참조는 빌드 단계에서 자동 검사한다 (`tools/build.py` 가 문서를 만든 뒤 `check_ids` 를 돌려 끊긴 참조가 있으면 실패로 끝난다).

1. 배경과 문제 정의

1.1 한 줄 문제

> **계획서는 Task를 잘게 나눠 주는데, 실행은 그중 하나씩만 한다.**

Claude Code + superpowers로 개발하면 흐름이 `brainstorming → writing-plans → 실행` 이다. 계획서는 Task 10여 개로 쪼개지고, 실행 스킬 `subagent-driven-development` 는 **Task 하나를 구현 서브에이전트에게 맡기고, 리뷰를 마친 뒤 다음 Task로 간다.**

1.2 문제가 생기는 지점

- **독립 Task도 순차로 돈다.** 관찰 사례: 이 프로젝트의 기획 산출물 계획(`docs/superpowers/plans/2026-10-20-convoy-planning-docs.md`)은 Task 11개 중 **6개(T3·T4·T5·T6·T7·T10)가 `Depends: T2` 하나만** 가져 같은 wave 3에 놓인다. 그래도 SDD로 실행하면 차례를 기다린다. 계획서 일반에서도 독립 Task가 이만큼 많다는 것은 가설 이며, D09에서 계획 2종으로 확인한다.
- **장부는 있는데 병렬이 없다.** SDD는 계획별 장부(`<workspace>/progress.md`)를 두고, 지휘 세션이 완료·판정 줄을 덧붙여 압축(compaction) 뒤의 복구 지도로 쓴다. 구현자는 자기 보고서 파일에 쓰고, 지휘 세션은 리뷰 지적을 직접 고치지 않는다(263~267행·408~409행) — 원문에서 관찰되는 역할 분담이다(보장된 단일 작성자 설계로 명시된 것은 아니다). 그런데 같은 원문이 구현 서브에이전트의 병렬 디스패치를 금지한다(282행). 기록과 역할 분담은 있는데 동시 실행이 없다.
- **사람이 메운 우회책은 사람이 관리해야 했다.** 작성자가 직접 시도한 방법은 세션 여러 개를 열고 md 파일 하나를 공유해 서로 소통시키는 것이었다. 동작은 했지만 동시 쓰기 충돌·대기 순서·죽은 세션을 사람이 지켜봐야 했다(작성자 경험).

근거 — superpowers 6.4.2 원문 확인 (2026-10-20)
설치본 `~/.claude/plugins/cache/superpowers-dev/superpowers/6.4.2/skills/` 기준.
· `subagent-driven-development/SKILL.md` — 8행: Task마다 새 구현 서브에이전트를 띄우고 Task마다 리뷰한다. 141~152행: 계획별 작업공간의 `progress.md` 를 장부로 만들고, 완료 줄로 재개 지점을 판정하며, 압축 뒤엔 장부와 `git log` 를 복구 지도로 믿는다. 263~267행: 구현자는 보고서 파일에 전체 보고를 쓴다. 408~409행: 지휘 세션은 리뷰 지적을 직접 고치지 않는다. 282행: "Never dispatch multiple implementation subagents in parallel (conflicts)."
· `dispatching-parallel-agents/SKILL.md` — 12~14행: 서로 독립적인 문제 영역(예: 원인이 다른 실패 여러 건)마다 에이전트 하나씩 동시에 보낸다. 42~45행·129~134행: 같은 파일·같은 자원을 건드리면 쓰지 않는다. 79~85행: 결과 검토·충돌 확인·통합은 보낸 쪽이 사후에 한다.

1.3 기획 질문

superpowers가 만든 계획서를 그대로 받아, 의존하지 않는 Task를 동시에 돌리면서도 main을 지키고, 실패를 놓치지 않을 수 있는가?

> Convoy는 superpowers가 만든 계획서를 받아, 의존하지 않는 Task를 여러 작업자에게 동시에 맡기고, 조율은 코드가 한다.

이름은 선단(船團)에서 왔다. 배 여러 척(작업자)이 따로 움직이되 호위함 하나(coordinator)가 순서와 안전을 맡는다. 이름이 곧 구조다.

1.4 포지셔닝 — 대체가 아니라 빈자리 메우기

선택지	판단
독자 프레임워크 (브레인스토밍~실행 전부)	✗ 잘 되는 부분까지 다시 만든다
superpowers 위에 얹는 애드온	✅ 실행 단계만 맡는다. 계획서 읽기는 어댑터로 분리해 다른 형식도 받을 수 있게 둔다 (ADR-06)
superpowers 수정 (포크·PR)	보류 — 실측으로 효과를 증명한 뒤 기능 제안/PR (9장 로드맵)

AX 관점의 메시지: 잘 쓰던 도구를 갈아엎지 않고, 막히는 지점 하나를 찾아 메운다.

2. 현재 대안과 한계

superpowers 두 스킬은 6.4.2 원문(1.2 근거)으로, Agent 도구는 Claude Code에서 직접 확인한 도구 설명(2026-10-20)으로 적었다. Workflow는 이 환경의 스킬 목록에서 존재만 확인했고 동작은 확인하지 않아 **가설**로 표시한다. md 공유 수작업은 작성자 경험이다. 시장 규모·사용자 수 같은 수치는 싣지 않는다.

기준	SDD 순차 (subagent-driven-development)	Agent 도구 수동 병렬 (dispatching-parallel-agents)	Workflow 가설	md 공유 수작업 (작성자 경험)	Convoy
병렬성	없음 — 원문이 구현 서브에이전트 병렬 디스패치를 금지 (282행)	있음 — 한 응답에 여러 호출을 내면 동시 실행. 대상은 원문상 독립 문제 영역	스크립트가 정한 만큼 가능할 것으로 추정	있음 — 세션 수만큼	있음 — 의존 그래프에서 실행 가능한 Task를 동시에
조율 자동화	지휘 세션이 장부·리뷰 루프를 운영	없음 — 충돌 확인·통합을 보낸 쪽이 사후에 (79~85행)	스크립트 작성자가 직접 짜야 할 것으로 추정. superpowers 계획서 해석은 확인 안 됨	없음 — 공유 md를 각 세션이 읽고 씀	코드가 담당 — 대기열·잠금·병합 순서 (ADR-01)
사람 개입 지점	원문상 4가지(비가역·보안·worktree 밖 부수효과·계획 붕괴)만 멈추고 나머지는 지휘 세션이 결정	지휘 세션과의 대화	확인 안 됨	상시 — 충돌·대기·죽은 세션을 사람이 관리	승인 대기 칸 (needs_human — 작업자 질문 · 재시도 소진 · 병합 실패)과 main 반영 승인
결정성	순서는 계획서 순. 판단(Ruling)은 지휘 세션이 내린다 (18~24행)	무엇을 병렬로 보낼지 지휘 세션이 판단 — 원문은 판단 흐름(독립인가 · 상태를 공유하는가, 16~45행)만 주고 결정은 보내는 쪽이 한다	스크립트 부분은 결정적일 것으로 추정	비결정 — 각 세션 LLM이 판단	조율 규칙은 결정적 코드, 테스트로 고정

기준	SDD 순차 (subagent-driven-development)	Agent 도구 수동 병렬 (dispatching-parallel-agents)	Workflow 가설	md 공유 수작업 (작성자 경험)	Convoy
main 보호	원문상 작업 전 격리 작업공간(worktree) 확인	Agent 도구에 worktree 격리 옵션이 있으나 병합은 수동	확인 안 됨	없음	integration 브랜치 경유 + 사용자 승인 (POL-05, POL-06)

정리: 병렬을 주는 대안(수동 병렬·md 공유)은 조율을 사람이나 LLM 판단에 맡기고, 조율을 갖춘 대안(SDD)은 병렬을 금지한다. Convoy는 **병렬성과 결정적 조율을 함께** 주는 칸을 노린다. Workflow와의 비교는 동작을 확인한 뒤 보완한다 조사 예정.

3. 대상 사용자 (페르소나)

가설 아래 두 페르소나는 기획자의 가설이다. 부록 A 가이드로 인터뷰해 부록 B에 기록하고, 결과에 따라 수정한다. **인터뷰 결과를 지어내지 않는다.**

P1. Claude Code로 혼자 제품을 만드는 개발자 가설

항목	내용
상황	계획서 하나에 Task 10~15개. 실행 중 대부분의 시간을 기다리며 보낸다
지금 하는 일	SDD로 순차 실행하거나, 세션을 여러 개 열어 손으로 나눠 맡긴다
원하는 것	독립 Task는 동시에 돌고, 자신은 판단이 필요할 때만 불려 오기
Convoy에서의 가치	`convoy check` 로 병렬 이득을 미리 보고(SN-01), 실행 중엔 상황판만 본다(SN-03)

P2. 2~5인 팀에서 AI 작업을 돌리는 리드 가설

항목	내용
상황	여러 작업을 동시에 AI에게 맡기되, 팀이 공유하는 main은 깨지면 안 된다
지금 하는 일	작업마다 브랜치를 따고 결과를 손으로 합친다
원하는 것	병합 순서와 테스트를 자동으로, main 반영은 자기 승인으로만
Convoy에서의 가치	integration 브랜치에 하나씩 병합·테스트하고, `convoy finish` 에서만 main에 반영한다(SN-08)

4. 핵심 가치

#	핵심 가치	구현 방식	근거
V1	**기다리지 않는다**	계획서의 `Depends` 로 의존 그래프를 만들고, 실행 가능한 Task를 작업자 여럿에게 동시에 맡긴다. 적혀 있지 않으면 순차로 본다	POL-01, POL-02, ADR-01
V2	**main은 지킨다**	결과는 `convoy/integration` 에 유효순위 순으로 한 번에 하나씩 병합하고 병합마다 테스트한다. main 반영은 전체 완료 후 사용자 승인으로만	POL-05, POL-06, CMD-10
V3	**조용히 실패하지 않는다**	조율은 결정적 코드가 맡고, Convoy 설계에서는 상태 파일(`state.json` · `BOARD.md`)을 coordinator 하나만 쓴다(ADR-01 — 관례가 아니라 구조로 강제). 환경 실패는 전체 일시정지, 상황판엔 마지막 갱신 경과 시간을 상시 표시	POL-11, POL-12, POL-18

원칙 — AI는 판단·제안, 조율은 코드. 대화·설명·의존 제안은 LLM이, 순서·우선권·잠금·병합은 결정적 코드가 맡는다. 그래야 조율 규칙을 전부 테스트로 증명할 수 있다. V2와 V3가 V1의 전제다 — 빨라져도 main이 깨지거나 실패를 놓치면 순차보다 못하다.

5. 사용 흐름

#	단계	명령 / 장면	담당	결과
1	계획서 작성	superpowers `writing-plans`	사람 + LLM	기존 형식 그대로. 선택 항목 `Depends` · `Resources` 두 줄(과 계획서 머리의 선택 `Test` 줄)만 추가
2	검사	`convoy check` (CMD-01 · SN-01)	코드	순환·없는 Task 참조·형식 오류면 실행 거부. 통과하면 wave 미리보기 + 이론상 최대 단축률
3	실행	`convoy run` (CMD-03 · SN-02, SN-03)	코드	coordinator 기동. 작업자별 worktree, 상황판 `BOARD.md` 갱신
4	승인	`convoy answer` / `approve` (CMD-05, CMD-06 · SN-04, SN-13)	사람	승인 대기 칸의 질문·충돌에 답한다. CLI든 앱 대화든 같은 명령을 거친다
5	마무리	`convoy finish` (CMD-10 · SN-08)	사람 승인 → 코드	integration → main 병합

- **두 입구, 하나의 작성자.** 터미널은 `convoy` CLI, Claude Code 안에서는 플러그인 `/convoy` 로 쓴다. 플러그인은 같은 CLI를 호출하고 결과를 대화로 풀어 줄 뿐 상태를 직접 고치지 않는다(ADR-05). 앱에서 "3번 왜 막혔어?"라고 물으면 대표 세션이 상황판·발신함·로그를 읽어 설명한다(SN-11, SN-12).
- **의존이 촘촘하면 실행 전에 안다.** 2단계에서 단축률을 숫자로 보여 주므로, 병렬 이득이 없는 계획은 돌리기 전에 걸러진다.
- **예시**: 이 기획 산출물 계획(계획 A)은 Task 11개를 `W1: T1 → W2: T2 → W3: T3 T4 T5 T6 T7 T10 → W4: T8 T9 → W5: T11` 의 다섯 wave로 나눈다(계획서의 Depends 표기 기준).

6. MVP 범위와 제외

6.1 포함

MVP 완료 조건은 **D02의 Must 요구사항 48건 전부**다. 아래 표는 영역별로 D02 요구사항을 묶은 것이며, Should · Could 항목은 우선순위를 함께 적었다 — MVP 범위 안에 두되 완료 조건은 아니다.

영역	포함 기능	D02 요구사항	근거
계획 읽기	superpowers `writing-plans` 형식 어댑터 1종 (`PlanAdapter` 뒤), `Depends` · `Resources` 선택 항목, 검사·wave 미리보기	REQ-001~REQ-007 (Must)	POL-01, POL-02
스케줄링	우선순위 + aging · 동률 처리 · 작업자 할당 · 실행 한도 · Task/Convoy 상태 전이 · 종료 상태	REQ-009~REQ-015, REQ-055 (Must)	POL-03, POL-04, POL-08
조율	outbox 메시지 7종 · 단일 작성자 · outbox 폴링 · 병합 대기열과 병합 후 테스트 · 독점 자원 대여 · lease · `.convoy/` 비추적	REQ-016~REQ-022, REQ-054 (Must) · REQ-023 긴 명령 전 progress (Should)	POL-05, POL-07, POL-16
사람 개입	`status` · 승인 대기 칸 · `answer` · `approve` · inbox 경유 · `finish`	REQ-024, REQ-026~REQ-030 (Must) · REQ-025 `status --watch` (Should)	POL-06, POL-18, POL-21
복구	`pause` · `resume` (복구 겸) · `stop` · PID 파일 · worktree 정리 시점	REQ-031~REQ-035 (Must)	POL-17, POL-19, POL-20
안전	비용 상한(draining) · 실패 분류 · 종료코드+본문 판정 · 작업자 권한 제한 · 도구 제한 실효 검증 · 프롬프트 주입 방어 · 스키마 위반 표시 · 침묵 감시	REQ-036~REQ-044 (Must)	POL-09~POL-16, POL-18
작업자	`claude -p` 헤드리스 1종 (`WorkerBackend` 뒤)	REQ-050 (Must)	ADR-04
입구	CLI + Claude Code 플러그인(CLI 래퍼) · 대화로 상태 질의 · 대화 속 승인 · CLI·앱 이어보기 · `annotate` 보조	REQ-045, REQ-046 (Must) · REQ-047~REQ-049 (Should) · REQ-008 `annotate` (Could, CLI 구현됨, 앱 흐름은 구현 전)	ADR-05, CMD-02
평가	기아 없음 성질 테스트 · LLM 없는 결정적 시험 계층 · E2E · `bench` 순차 대 병렬 비교	REQ-052, REQ-053 (Must) · REQ-056 (Should) · REQ-051 `bench` (Should — 계획 C에서 D09 실측과 함께)	CMD-11

6.2 제외 (향후 계획)

제외 항목	제외 이유	다시 검토할 조건
MCP 허브	stdio 방식은 클라이언트마다 서버가 따로 떠 단일 작성자가 깨지고, HTTP 데몬은 인프라가 늘어난다 (ADR-02)	파일 기반 허브 앞단에 붙일 수요가 생길 때
웹 대시보드	쓰던 도구 안에 녹이는 것이 AX 취지에 맞다 (ADR-05)	CLI·앱 상황판으로 부족하다는 인터뷰 결과가 반복될 때
superpowers PR	효과를 증명하기 전에 상류를 고치지 않는다 (1.4)	D09 실측에서 성공 지표 달성 후

제외 항목	제외 이유	다시 검토할 조건
다른 계획 형식 어댑터	1차는 superpowers 형식 하나. `PlanAdapter` 인터페이스만 남긴다	다른 계획 형식 사용자가 확인될 때

7. 성공 지표

목표치 아래 수치는 구현 전 **목표**이며 실측이 아니다. 계획 C에서 `convoy bench` (CMD-11)로 처음 측정하고, 결과는 **D09 실측 리포트**에 싣는다. 효과가 없는 경우도 정직하게 싣는다.

실측 결과는 D09 실측 리포트에 있다(아래 목표치 표는 구현 전 값 그대로 둔다).

#	지표	정의	목표 목표치	측정 방법 (D09)
M1	폭 넓은 계획 소요시간 단축	(순차 소요시간 – 병렬 소요시간) / 순차 소요시간	**30% 이상 단축**	폭이 넓은 계획을 작업자 1(순차) / 작업자 3(병렬)으로 각 3회 이상 실행해 총 소요시간 비교. LLM은 실행마다 흔들리므로 1회 비교로 결론 내지 않는다. 대조군으로 의존이 깊은 계획도 같이 잰다
M2	main 오염	사용자 승인(`convoy finish`) 없이 main에 들어간 커밋, 또는 테스트가 깨진 채 main에 반영된 병합	**0건**	모든 bench 실행 전후 main 커밋 목록을 대조하고, 반영된 병합마다 integration 테스트 결과를 확인
M3	조용한 실패	작업자 실패·죽음·환경 실패가 일어났는데 상황판(승인 대기 칸·일시정지 표시·마지막 갱신 경과)에 드러나지 않은 경우	**0건**	대본대로 움직이는 가짜 작업자로 실패·lease 만료·환경 실패를 주입하는 통합 테스트 + bench 중 실제 실패 사례를 상황판 기록과 대조

보조 지표 (목표 없이 기록): 비용 · 병합 충돌 수 · 재작업 수 · 최종 테스트 통과 여부. 비용은 금액(USD — 작업자 호출의 `total_cost_usd` 합)으로 기록한다(POL-09, 2026-10-20 `claude -p` 1회 실측으로 필드 확인). 구독 요금제에서 이 값이 실제 청구액인지는 구현 1단계 확인 필요.

M2·M3를 M1과 나란히 둔 이유: 단축률만 보면 병합을 대충 하거나 실패를 숨겨도 지표가 오른다. 빨라진 만큼 위험해지지 않았다는 증거가 함께 있어야 단축률을 믿을 수 있다.

8. 리스크와 대응

#	리스크	영향	대응	관련
R1	**병합 충돌 증가** — 동시에 돈 Task가 인접 영역을 고침	재작업·대기 증가로 단축률 잠식	`Files` 가 겹치면 동시 실행하지 않는다. 병합은 integration에 하나씩, 충돌·테스트 실패 시 그 병합을 되돌리고 사람에게 넘긴다 (`needs_human` , ST-T-08). 충돌 수를 D09에 기록	POL-02, POL-05
R2	**비용 증가** — 작업자마다 컨텍스트가 따로 든다	순차보다 비싸질 수 있음	실행 전체 비용 상한을 넘으면 새 작업자를 띄우지 않고 진행 중인 것만 마무리(`draining` , ST-C-03). 동시 작업자 수 상한. 비용은 D09에서 순차와 나란히 공개	POL-08, POL-09

#	리스크	영향	대응	관련
R3	**CLI 동작 변경** — `claude -p` 의 출력 형식·종료코드·도구 제한 옵션이 버전에 따라 바뀜	실패를 성공으로 오판, 작업자 권한 이탈	종료코드와 출력 본문을 둘 다 본다. 도구 제한은 미끼 파일로 뚫어 보고 확인해 테스트로 고정. 작업자는 `WorkerBackend` 뒤에 숨겨 교체 가능하게 둔다. 현재 동작 자체는 아직 실측 전 구현 1단계 확인 필요	POL-12, POL-14, ADR-04
R4	의존 표기 누락·오기	의존 Task가 먼저 돌아 결과가 틀림	`Depends` 가 없으면 직전 Task 의존(순차)으로 본다 — 기존 계획서를 넣어도 깨지지 않는다. `annotate` 는 제안만 하고 사람이 승인한 것만 기록	POL-01, CMD-02
R5	페르소나 가설이 틀림	잘못된 우선순위	인터뷰(부록 A·B)로 검증 후 범위 재조정	—

9. 로드맵

단계	내용	산출물	상태
계획 A — 기획	문제 정의, 요구사항, 정책, 규격, IA, 대화형 UX, 기능명세, 시스템·AI 설계	D01~D08	완료
계획 B — 구현	core(planner · scheduler · arbiter · supervisor · board) · CLI · 플러그인 · 테스트	동작하는 `convoy` (배포 패키지 `convoy-plan`)	완료
계획 C — 실측과 마무리	`convoy bench` 로 순차 대 병렬 실측, 요약본	D09 실측 리포트, D10 요약본 + README	완료
향후	실측 결과를 근거로 superpowers 기능 제안/PR, 다른 계획 형식 어댑터	—	실측 후 판단

계획 C 는 Convoy 로 돌리지 않았다 — 작업자 허용 목록으로는 PDF 빌드와 .venv 가 필요한 Task 를 못 하고 실측 Task 는 직렬이어서다. 자기 자신을 돌리는 시연은 bench 의 wide 계획이 대신했다.

부록 A. 사용자 인터뷰 가이드

목적: 페르소나 가설(3장)과 핵심 가치 V1~V3가 실제 수요인지 확인한다. **대상**: 최근 3개월 안에 Claude Code 등 AI 코딩 에이전트로 계획서 기반 작업을 돌려 본 사람 3~5명 (P1·P2 유형 각 1명 이상). **시간**: 30분 · 녹음은 동의 시에만.

#	단계	질문	확인하려는 것
1	현재 행동	마지막으로 AI에게 여러 단계짜리 작업을 맡긴 때를 떠올려 주세요. 어떻게 나눠서 맡기셨나요?	현재 실행 방식
2	현재 행동	그 작업이 도는 동안 무엇을 하고 계셨나요? 얼마나 자주 들여다보셨나요?	대기 시간의 실체
3	문제 검증	서로 상관없는 작업을 동시에 돌려 본 적이 있나요? 어떻게 하셨나요?	병렬 우회책 유무
4	문제 검증	동시에 돌렸을 때 결과를 합치다 문제가 생긴 적이 있나요? 무엇이었나요?	병합 충돌 리스크(R1)
5	문제 검증	AI 작업이 중간에 멈추거나 실패했는데 한참 뒤에야 안 적이 있나요?	조용한 실패(V3) 수요
6	문제 검증	AI가 만든 변경이 main(공유 브랜치)에 바로 들어가는 것에 대해 어떻게 하고 계세요?	main 보호(V2) 수요
7	반응	(`convoy check` 결과 장면을 보여 주며) 실행 전에 이 숫자를 보면 무엇을 판단하시겠어요?	사전 단축률 표시의 쓸모
8	반응	사람 판단이 필요할 때 어디서 알림을 받고 싶으세요? (터미널 / 대화창 / 기타)	입구 선택(ADR-05) 근거
9	우려	여러 작업자를 동시에 돌린다면 가장 걱정되는 점은 무엇인가요? (비용·품질·통제 등)	리스크 우선순위
10	마무리	계획서 하나에 Task가 보통 몇 개쯤 되나요? 그중 서로 기다릴 필요가 없는 건 몇 개쯤인가요?	병렬 이득의 크기

진행 원칙: Convoy를 먼저 설명하지 않는다(1~6번은 제품 언급 없이) · "좋아요?"처럼 답을 유도하는 질문을 하지 않는다 · 의견보다 과거의 실제 행동을 묻는다.

부록 B. 인터뷰 결과 기록 양식

인터뷰 후 직접 채운다. 비어 있는 상태가 정상이다.

항목	참여자 1	참여자 2	참여자 3	참여자 4	참여자 5
일자 / 유형 (P1·P2·기타)					
현재 실행 방식 (Q1)					
대기 중 행동·확인 빈도 (Q2)					
병렬 우회책 (Q3)					
병합 문제 경험 (Q4)					
늦게 안 실패 경험 (Q5)					
main 반영 방식 (Q6)					
사전 단축률 반응 (Q7)					

항목	참여자 1	참여자 2	참여자 3	참여자 4	참여자 5
선호 알림 위치 (Q8)					
가장 큰 우려 (Q9)					
Task 수 / 독립 Task 수 (Q10)					
가설 판정					

판정 기준: 지지 / 부분 지지 / 반박 중 하나로 적는다. **종합 (작성 후)**: 페르소나 수정 사항 · 우선순위 변경 · 지표 목표 조정